Are Tone Marks the Weak Point of Open Thai OCR? A Locked-Split Study of Eight Models

ธนกฤต เสมอหน้า[1] ธนวัฒน์ สุริยะ[1] ผศ.ดร. นนท์ คนึงสุขเกษม[2]

คณะเทคโนโลยีสารสนเทศ สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง

เลขที่ 1 ซอยฉลองกรุง 1 แขวงลาดกระบัง เขตลาดกระบัง กรุงเทพมหานคร 10520

67070060@kmitl.ac.th, 67070236@kmitl.ac.th, nont@it.kmitl.ac.th

[1] นักศึกษา [2] ผู้ช่วยศาสตราจารย์

บทคัดย่อ

Vision-Language Model (VLM) แบบเปิดสำหรับ OCR ภาษาไทยอ่านหน้าเอกสารได้ถูกต้องเกือบทั้งหน้าแล้ว แต่สระและวรรณยุกต์ของไทยมีขนาดเล็ก และเครื่องหมายเพียงตัวเดียวก็เปลี่ยนความหมายของคำได้ งานนี้จึงถามว่าเครื่องหมายเหล่านี้เป็นจุดอ่อนของโมเดล OCR ภาษาไทยแบบเปิดหรือไม่ และหายไปที่ใด เราเสนอคะแนนที่วัดเครื่องหมายโดยไม่สนลำดับการอ่าน หักคะแนนข้อความที่เขียนเกิน และไม่ให้คะแนนกับข้อความที่ซ้ำจากการวนซ้ำ เราพัฒนาการวิเคราะห์บนชุด calibration ของ ThaiOCRBench แล้วยืนยันบนชุด locked 352 ข้อ ซึ่งเปิดอ่านเพียงครั้งเดียว ด้วยโมเดลแบบเปิดแปดตัว คำตอบคือส่วนใหญ่ไม่ใช่ ในทั้งแปดโมเดล ข้อความที่ไม่ปรากฏใน output ทำให้เสีย mark มากกว่าการอ่าน mark ผิดหลังอ่านพยัญชนะถูก 4.7 ถึง 15.8 เท่า เมื่อได้รับข้อความก่อนหน้าที่ถูกต้อง Typhoon OCR 1.5 (2B) และ Pathumma เลือกวรรณยุกต์ถูกราว 99% โดยภาพช่วยเพียงเล็กน้อย ส่วน Qwen3-VL-4B รู้ mark เหล่านี้ได้ดีเท่ากัน แต่อ่านผิดบางตัวเมื่อเขียนหน้าเอง ข้อค้นพบนี้จึงเป็นจริงเพียงบางส่วนสำหรับโมเดลทั่วไป การหยุดเมื่อตรวจพบการวนซ้ำเพิ่ม mark F1 ใน 12 จาก 13 เซลล์ (โมเดลและงาน) ที่โมเดลวนซ้ำราว 5% ของหน้าขึ้นไป เพิ่มขึ้น 4.8 ถึง 35.6 จุด และเปลี่ยน output ที่จบเองเพียง 1 จาก 2,400 รายการ เซลล์ที่ไม่ผ่านคือ Pathumma บนหน้าเต็ม ซึ่งวนซ้ำน้อย ส่วน repetition penalty ทำให้ F1 ของหน้าที่ไม่วนซ้ำลดลง และการหักความรู้ภาษาออกทำให้วรรณยุกต์ที่ถูกอยู่แล้วหายไป ผลทั้งหมดมาจาก benchmark เดียว Typhoon มาจากกลุ่มที่เผยแพร่ benchmark นี้ และไม่ระบุข้อมูลฝึกของ Pathumma เราจึงไม่จัดอันดับสองโมเดลนี้

คำสำคัญ—OCR ภาษาไทย, vision-language model, วรรณยุกต์และสระ, การวนซ้ำ, การ decode โดยไม่ต้อง train เพิ่ม

1. Introduction

อักษรไทยเขียนพยัญชนะเป็นแนวหลัก โดยมีสระและวรรณยุกต์วางซ้อนอยู่ด้านบนและด้านล่าง เราเรียกอักขระซ้อนเหล่านี้ว่า mark ได้แก่ สระบน สระล่าง และวรรณยุกต์ (ระบุไว้ในหัวข้อ 3) mark ตัวเดียวก็เปลี่ยนความหมายและเสียงของคำได้ เช่น “ข่าว” กับ “ข้าว” การถอดข้อความที่ทำ mark หายหรือสลับตัวจึงผิดสำหรับ

ผู้อ่าน แม้ตัวอักษรอื่นเกือบทั้งหมดจะถูกต้อง แต่การให้คะแนน OCR ภาษาไทยมักใช้ตัวชี้วัดของข้อความทั้งก้อน เช่น ความทับซ้อนของ n-gram และ normalized edit distance ใน ThaiOCRBench [1] หรือ character error rate ในงานอื่น ตัวชี้วัดเหล่านี้บอกไม่ได้ว่าปัญหาอยู่ที่ mark หรือไม่ และคะแนนที่เทียบทั้งลำดับยังลงโทษโมเดลที่อ่านคอลัมน์ของหน้าในลำดับต่างจากเฉลยด้วย

ปัจจุบันมี vision-language model (VLM) แบบเปิดสำหรับ OCR ภาษาไทยแล้ว [2–4] เมื่อโมเดลเหล่านี้ทำ mark ผิด มีคำอธิบายที่เป็นไปได้สามข้อ หลักฐานในภาพของ mark ขนาดเล็กอาจอ่อน ความรู้ภาษาของโมเดลอาจทับหลักฐานในภาพ หรือ mark นั้นอาจไม่ปรากฏใน output เลย เพราะโมเดลข้ามบรรทัด หาข้อความไม่เจอ หรือติดการวนซ้ำ แต่ละข้อต้องแก้ต่างกัน คืออ่านซ้ำหรือขยายภาพเมื่อหลักฐานในภาพอ่อน หักความรู้ภาษาออกแบบ contrastive decoding [5, 6] เมื่อความรู้ภาษาทับหลักฐาน และแก้วิธี decode หรือการหาตำแหน่งข้อความเมื่อข้อความหายจาก output จาก output อย่างเดียวเราบอกไม่ได้ว่าภายในโมเดลเกิดอะไรขึ้น จึงใช้คำว่าบรรทัดนั้น “ไม่ปรากฏใน output” และไม่ได้อ้างว่าโมเดลมองไม่เห็น

เราศึกษาคำถามนี้บน ThaiOCRBench ด้วย Typhoon OCR 1.5 และ checkpoint แบบเปิดอีกเจ็ดตัว ในงานอ่านทั้งหน้าและงานอ่านข้อความสั้น เราพัฒนาการวิเคราะห์บนชุด calibration แล้วยืนยันบนชุด locked ซึ่งเราเปิดอ่านเพียงครั้งเดียว งานนี้มีส่วนร่วมสามข้อ

1. คะแนน mark ที่ไม่สนลำดับการอ่าน (หัวข้อ 3) ให้คะแนนสระบน สระล่าง และวรรณยุกต์ทีละบรรทัดของเฉลย ไม่ว่าบรรทัดนั้นจะอยู่ตรงไหนของ output ข้อความที่เขียนเกินถูกหักคะแนน และสำเนาที่เกิดจากการวนซ้ำไม่ได้คะแนน

2. การวินิจฉัยว่า mark หายไปที่ใด (หัวข้อ 4 และ 6) บนชุด locked ในทั้งแปดโมเดล บรรทัดที่ไม่ปรากฏใน output ทำให้เสีย mark มากกว่า mark ที่อ่านผิดหลังอ่านพยัญชนะถูก 4.7 ถึง 15.8 เท่า เมื่อได้รับข้อความก่อนหน้าที่ถูกต้อง Typhoon และ Pathumma เลือกวรรณยุกต์ถูกราว 99% โดยภาพช่วยเพียงเล็กน้อย Qwen3-VL-4B แสดงว่าข้อนี้ไม่เป็นจริงกับทุกโมเดล เพราะรู้ mark ได้ดีเท่ากัน แต่ก็อ่านบางตัวผิดตอนเขียนหน้าเอง

3. การประเมินวิธี decode ที่ไม่ต้องฝึกโมเดลเพิ่ม (หัวข้อ 5 และ 6) บนชุด locked การหยุดเมื่อตรวจพบการวนซ้ำเพิ่ม mark F1 ใน 12 จาก 13 เซลล์ที่ลงทะเบียนไว้ และเปลี่ยน output ที่จบเองเพียง 1 จาก 2,400 รายการ ส่วน repetition penalty ทำให้ F1 ของหน้าที่ไม่วนซ้ำลดลง และการหักความรู้ภาษาออกทำให้วรรณยุกต์ที่ถูกอยู่แล้วหายไป

สำหรับโมเดลที่เราศึกษา mark ที่โมเดล OCR ภาษาไทยทำหายส่วนใหญ่หายก่อนจะถึงตัว mark วิธี decode และการหาตำแหน่งข้อความจึงสำคัญกว่าการทำให้มองวรรณยุกต์ชัดขึ้น

2. Related Work

OCR ภาษาไทยด้วย VLM ThaiOCRBench [1] ประเมิน VLM ทั้งแบบปิดและแบบเปิดบนงานเอกสารภาษาไทย รวมถึงงานอ่านทั้งหน้าและงานอ่านข้อความสั้น โดยใช้ตัวชี้วัดที่อิงความทับซ้อนของ n-gram และ edit distance ได้แก่ BLEU, METEOR, F1 และ normalized Levenshtein similarity ส่วนงานอ่านข้อความสั้นใช้ ANLS บทนำของงานนี้ระบุอักขระซ้อนเป็นลักษณะของภาษาไทยที่ชุดข้อมูลภาษาอังกฤษเป็นหลักไม่ครอบคลุม แต่ไม่มีคะแนนที่แยกเครื่องหมายออกมา หลังจากนั้นมี OCR ภาษาไทยแบบเปิดออกมาหลายตัว Typhoon OCR [2] เวอร์ชัน 1.5 เป็น full fine-tune ของ Qwen3-VL-2B [7] และผู้พัฒนาระบุว่า infographic ยัง

ต้องปรับปรุง Pathumma-llm-vision 3.0 [3] เป็น fine-tune ของ Qwen3-VL-2B อีกตัว ส่วน wayu-paxa-ocr-zero [4] fine-tune PaddleOCR-VL [8] ด้วยหน้าเอกสารไทยสังเคราะห์ ทุกงานรายงานความแม่นยำรวม ส่วนงานของเราถามว่าเครื่องหมายหายไปที่ใด

Diacritic ใน OCR อักษรที่มี diacritic มากเป็นที่ทราบกันว่ายากสำหรับ OCR มีงานเสนอการแก้ข้อความหลัง OCR สำหรับ diacritic ภาษาเวียดนาม [9] และงานประเมิน OCR ภาษาอาหรับแยกข้อความที่มี diacritic เป็นอีกเงื่อนไขหนึ่ง [10] เราไม่พบงานที่ให้คะแนนเครื่องหมายภาษาไทยแยกจากลำดับการอ่านและข้อความที่เขียนเกิน

การวนซ้ำและการตกหล่น การ decode ที่เลือกคำตามความน่าจะเป็นสูงสุดทำให้ข้อความวนซ้ำ [11] และโมเดล OCR เอกสารก็มีพฤติกรรมเดียวกัน Nougat รายงานหน้าที่โมเดลพิมพ์ประโยคเดิมซ้ำ และหยุด generate เมื่อสัญญาณความแปรปรวนของ logit ลดลง [12] olmOCR สุ่มใหม่ด้วย temperature ที่สูงขึ้น [13] pipeline ของ PaddleOCR-VL ตัดข้อความซ้ำหลัง decode เสร็จ [14] และโค้ด DeepSeek-OCR ที่เผยแพร่ห้ามการซ้ำของ n-gram ภายในหน้าต่างหนึ่ง [15, 16] CC-OCR นับการวนซ้ำเป็น hallucination แบบหนึ่งของ OCR [17] งานเหล่านี้มองการวนซ้ำเป็นสิ่งที่ต้องตรวจจับหรือหลีกเลี่ยง ส่วนเราวัดว่าวิธีแก้แต่ละแบบทำให้เครื่องหมายในหน้าที่ไม่วนซ้ำเสียไปเท่าใด หนึ่งในวิธีที่วัดคือ repetition penalty ของ CTRL [18] ซึ่ง package OCR บางตัวเปิดไว้เป็นค่าเริ่มต้น

ความรู้ภาษาและ decoding ที่ไม่ต้องฝึกเพิ่ม decoding ที่ไม่ต้องฝึกเพิ่มลด hallucination ของ VLM โดยให้น้ำหนักกับสิ่งที่ภาพรองรับมากกว่าสิ่งที่ความรู้ภาษาอย่างเดียวจะทำนาย VCD [5] เทียบการแจกแจงของ output ระหว่างภาพจริงกับภาพที่ถูกบิดเบือน M3ID [6] เพิ่มอิทธิพลของภาพเหนือความรู้ภาษา และ PAI [19] เพิ่ม attention ที่ให้แก่ token ของภาพและหัก logit ของอินพุตที่มีแต่ข้อความ วิธีเหล่านี้สมมติว่าความรู้ภาษาเป็นต้นเหตุ แต่สำหรับวรรณยุกต์ไทยเราพบว่าความรู้ภาษามักถูกอยู่แล้ว วิธีที่เราใช้เป็นการเขียนใหม่ จึงเรียกว่า M3ID-form และ PAI-style ส่วน speculative decoding [20] ที่ใช้ prompt lookup เป็นตัวร่าง [21] ทำให้ decode เร็วขึ้น และ output ไม่เปลี่ยนเมื่อคำนวณแบบแม่นยำ

3. Mark Score and Setup

เครื่องหมายที่เราวัดเรียกว่า mark ได้แก่ สระบนหกตัว (*ั ิ ี ึ ื ็*) สระล่างสองตัว (*ุ ู*) และวรรณยุกต์สี่ตัว (*่ ้ ๊ ๋*) เฉลยของหน้าเต็มมีหลายบรรทัด และโมเดลอาจอ่านคอลัมน์ของหน้าในลำดับต่างจากเฉลย การจัดแนวแบบ global จึงนับ mark ที่อ่านถูกว่าเป็นข้อผิด

นิยาม เราพิจารณาบรรทัดของเฉลยที่ยาวอย่างน้อยแปดตัวอักษร เรียงจากยาวไปสั้น แต่ละบรรทัดหาช่วงของ output ที่ตรงกับบรรทัดนั้นที่ character error rate ต่ำกว่า 40% โดยไม่นับอักขระที่ถูกจองไปแล้ว จากนั้นให้คะแนน mark ที่ตรงกัน และจองช่วงนั้นไว้ เพื่อให้ช่วงเดียวของ output ไม่ให้คะแนนสองบรรทัด บรรทัดที่เหลือถูกจัดแนวแบบ global ตามลำดับของเฉลยกับ output ที่ตัดอักขระที่จองไว้ออกทั้งหมด หากไม่มีขั้นตอนที่สองนี้ mark ในบรรทัดที่อ่านคลาดเคลื่อนจะหลุดไป และเวอร์ชันแรกของเราไม่ผ่านการตรวจกับโมเดลตั้งต้น Recall คือ mark ที่ได้คะแนนหารด้วย mark ในเฉลย ส่วน precision คือ mark ที่ได้คะแนนหารด้วย mark ทั้งหมดใน output ซึ่งทำให้ข้อความที่เขียนเกินถูกหักคะแนน ส่วน F1 คำนวณจากผลรวมของทุกข้อ

แบบที่กันการวนซ้ำ ในแบบนี้ คะแนนคำนวณหลังย่อทุกช่วงที่วนซ้ำให้เหลือหนึ่งสำเนา ช่วงที่วนซ้ำคือหน่วยยาว 4 ถึง 400 ตัวอักษรที่มีตัวอักษรอย่างน้อยหนึ่งตัวและซ้ำกันเหมือนกันทุกตัวอย่างน้อยแปดครั้ง หรือหกครั้งสำหรับหน่วยที่ยาว 50 ตัวอักษรขึ้นไป ส่วน precision ยังนับ mark ทุกตัวของ output เต็ม สำเนาจึงถูกหักคะแนนแต่ไม่ได้ recall เราเพิ่มแบบนี้หลังจากผู้ร่วมงานพบว่า alignment จับคู่สำเนาของหน่วยเดียวที่ซ้ำ 434 ครั้งเข้ากับพยางค์ทั่วไปในเฉลย ถ้าไม่มีการวนซ้ำ คะแนนทั้งสองแบบเท่ากัน

การตรวจสอบ การอ่านหน้าเดียวกันสองรอบต่อกันได้คะแนนต่ำกว่าการอ่านรอบเดียวที่ดีกว่ามาก recall ของโมเดลตั้งต้นต่างจาก global alignment ไม่เกิน 0.3 จุดในทุกเซลล์ และ recall ของ Typhoon บนหน้าเต็มเพิ่มขึ้น 1.6 ถึง 1.8 จุดเมื่อไม่หักคะแนนเรื่องลำดับ การหักเรื่องลำดับอย่างเดียวทำให้ mark F1 ของ Typhoon บนหน้าเต็มลดจาก 94.9 เหลือ 93.1 สำหรับ prompt BQ (จาก 95.3 เหลือ 93.7 สำหรับ TC ซึ่งนิยามในหัวข้อถัดไป) โมเดลตั้งต้นได้รับผลกระทบน้อยมาก

ข้อมูล เราใช้ชุด test ของ ThaiOCRBench [1] สองงาน คืองานอ่านทั้งหน้าและงานอ่านข้อความสั้น ชุด calibration สุ่มแบบกำหนด seed ไว้แบ่งชั้นตามงานและหมวด มีงานอ่านทั้งหน้า 69 ข้อ และงานอ่านข้อความสั้น 109 ข้อ ส่วนที่เหลือของ benchmark เป็นชุด locked มีงานอ่านทั้งหน้า 128 ข้อ และงานอ่านข้อความสั้น 224 ข้อ เราไม่เปิดชุดนี้จนกว่ากฎทุกข้อในหัวข้อ 4 และ 5 จะถูกกำหนดแล้ว และเปิดอ่านเพียงครั้งเดียวเพื่อการยืนยันในหัวข้อ 6 เรารายงาน output และคะแนนรายข้อ และไม่เผยแพร่ภาพของ benchmark

โมเดล เราประเมิน Typhoon OCR 1.5 (2B) โมเดลตั้งต้น Qwen3-VL-2B, Pathumma-llm-vision 3.0, Qwen3-VL-4B, Qwen-SEA-LION-v4-4B-VL [22], DeepSeek-OCR-2 [23] และโมเดลขนาด 0.9B ที่ออกแบบให้อ่านทีละกรอบข้อความ คือ PaddleOCR-VL-1.6 และ wayu-paxa-ocr-zero (เรียกว่า Wayu) โดยนำสองตัวหลังมาอ่านทั้งหน้าด้วย ซึ่งเกินกว่าที่ออกแบบไว้ หัวข้อ 4 และ 5 รายงานหกโมเดลแรก ส่วนสองตัวสุดท้ายเข้ามาในหัวข้อ 6 DeepSeek-OCR-2 ใส่ GPU T4 แบบ fp32 ไม่ได้ ตัวเลขทั้งหมดของโมเดลนี้จึงเป็น fp16 และมาจาก runner ของมันเอง ทุก checkpoint ระบุ revision ไว้ และเราใช้ revision ของ Typhoon ที่เผยแพร่ภายใต้ Apache-2.0 อย่างเดียว ค่า revision, seed และรหัสการรันบันทึกไว้ใน repository ของโครงการ

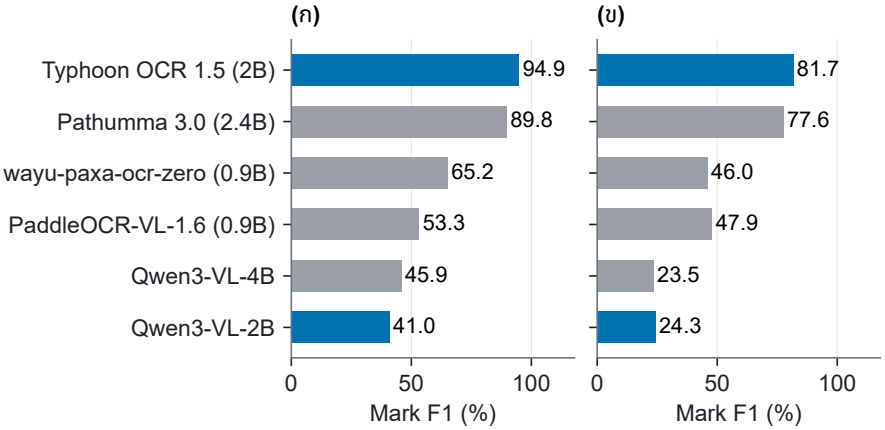
Prompt และการ decode BQ คือคำถามของ benchmark ในแต่ละข้อ ส่วน TC คือ prompt ที่ model card ของ Typhoon แนะนำ Paddle และ Wayu ใช้ prompt `OCR:` ส่วน Pathumma และ DeepSeek-OCR-2 ใช้ prompt ใน model card ของตัวเอง การ decode ใช้ greedy สร้าง token ใหม่ไม่เกิน 3,072 ตัว ด้วย fp16 บน GPU Kaggle T4 แต่ละผลมาจากการรันครั้งเดียว output แบบ greedy กับ output ที่ตัดที่การวนซ้ำมาจากการรันเดียวกัน จึงเป็นการเทียบแบบจับคู่ output ถูกแปลงเป็นข้อความธรรมดาก่อนให้คะแนน สำหรับ Typhoon BQ เป็น prompt หลัก และ TC เป็น prompt รอง

สถิติ ช่วงความเชื่อมั่นคือ bootstrap 95% ระดับรายข้อ หัวข้อ 4 และ 5 มาจากชุด calibration ผลจึงเป็นผลเบื้องต้น และกฎ decode บางข้อถูกกำหนดหลังจากเห็นชุดนี้แล้ว ซึ่งเราระบุไว้ในจุดที่เกี่ยวข้อง ส่วนหัวข้อ 6 ลงทะเบียนไว้ก่อนให้คะแนน output ของชุด locked ใดๆ และใช้ bootstrap 10,000 รอบกับการแก้ Holm หนึ่งชุด

4. Where Marks Are Lost

หกโมเดล บนชุด calibration Typhoon ได้ mark F1 94.9 บนหน้าเต็ม และ 81.7 บนงานอ่านข้อความสั้น (รูปที่ 1 และตารางที่ 1) Pathumma-llm-vision 3.0 [3] ซึ่งเป็น fine-tune ภาษาไทยตัวที่สองจากโมเดลตั้งต้นเดียวกัน ได้ใกล้เคียงกัน เมื่อใช้ prompt ของ model card เอง ได้ 89.8 และ 77.6 ส่วนต่างจาก Typhoon คือ −5.1 จุดบนหน้าเต็ม (ช่วง −14.3 ถึง +1.6) และ −4.1 บนงานอ่านข้อความสั้น (ช่วง −15.1 ถึง +8.0) เราจึงบอกไม่ได้ว่าตัวใดดีกว่า ทั้งสองตัวสูงกว่าโมเดลตั้งต้นมาก คือ 48.9 และ 53.3 จุด ส่วน checkpoint อีกสามตัวต่ำกว่า Typhoon มาก Wayu ซึ่งดีที่สุดในสามตัวนี้ต่ำกว่า 29.6 จุดบนหน้าเต็ม (ช่วง 19.2 ถึง 39.9) การเพิ่มขนาดโมเดลตั้งต้นเป็นสองเท่าเปลี่ยน F1 บนหน้าเต็มเพียง +5.0 (ช่วง −4.0 ถึง +14.1) และอัตราผิดของวรรณยุกต์ยังอยู่ราว 9.4 ถึง 9.9% ค่ามัธยฐานของ character error rate ต่อหน้าของ Typhoon คือ 2.8%

การเทียบนี้มีข้อควรระวังสองข้อ โมเดลถูกเทียบตามที่เผยแพร่ ซึ่งจำนวน visual token ไม่เท่ากัน และเราตัดความเป็นไปได้ที่ข้อมูลซ้อนกับ benchmark ไม่ได้สำหรับสองโมเดลที่ดีที่สุด model card ของ Pathumma รายงานคะแนน ThaiOCRBench และไม่ระบุที่มาของข้อมูลฝึก 377K ตัวอย่าง ส่วน Typhoon มาจากกลุ่มที่เผย



รูปที่ 1 Mark F1 บนชุด calibration (ก) งานอ่านทั้งหน้า (ข) งานอ่านข้อความสั้น ใช้ prompt BQ ยกเว้นโมเดล 0.9B สองตัวที่ใช้ `OCR:` และ Pathumma ที่ใช้ prompt ของ model card

ตารางที่ 1 Mark F1 อัตราผิดของวรรณยุกต์หลังอ่านพยัญชนะถูก และสัดส่วน output ที่ชนขีดจำกัด token (%) Pathumma ใช้ prompt ของ model card

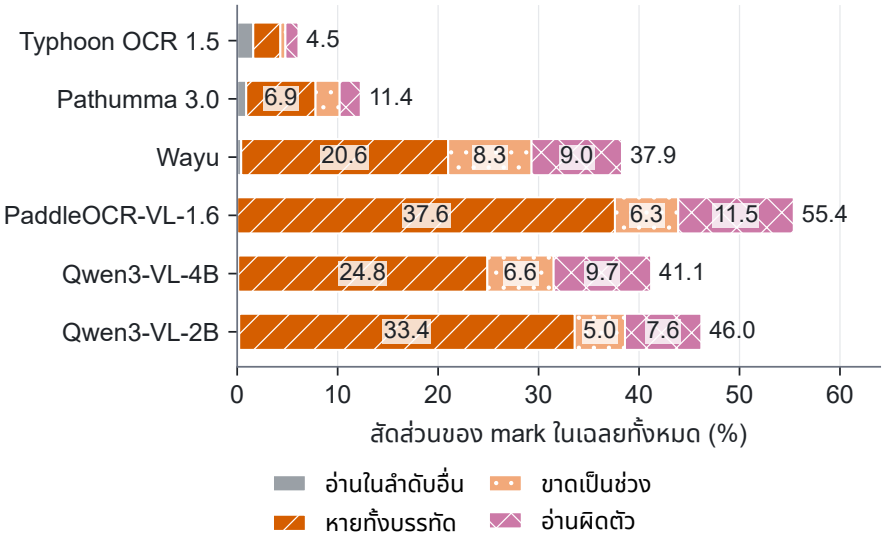
	อ่านทั้งหน้า			อ่านข้อความสั้น	
โมเดล	F1	ผิดวรรณยุกต์	ชนขีดจำกัด	F1	ชนขีดจำกัด
Typhoon OCR 1.5	**94.9**	0.65	2.9	**81.7**	0.9
Pathumma 3.0	89.8	0.99	5.8	77.6	1.8
Wayu	65.2	5.27	26.1	46.0	20.2
PaddleOCR-VL-1.6	53.3	14.57	39.1	47.9	14.7
Qwen3-VL-4B	45.9	9.89	27.5	23.5	11.9
Qwen3-VL-2B	41.0	9.41	43.5	24.3	14.7

แพร่ ThaiOCRBench เราจึงเรียก Typhoon ว่าแข็งที่สุดในบรรดาโมเดลที่เราวัด และใกล้เคียงกับ Pathumma เท่านั้น

ในงานอ่านข้อความสั้น สองโมเดลพลาดต่างกัน Pathumma พบ mark ที่ขอได้มากกว่า (recall สูงกว่า 14.3 จุด) แต่มักถอดทั้งภาพแทนที่จะถอดเฉพาะช่วงที่ถาม ทำให้ precision ต่ำลง

สำเนาของการวนซ้ำเคยได้ recall คะแนนแบบกันการวนซ้ำเปลี่ยน Typhoon ไม่เกิน 0.2 จุด (F1 จาก 94.9 เหลือ 94.8 และ recall 96.1 เท่าเดิม) ส่วนโมเดลตั้งต้น (BQ) บนหน้าเต็ม recall ลดจาก 65.0 เหลือ 54.6 precision ลดจาก 29.9 เหลือ 25.1 และ F1 ลดจาก 41.0 เหลือ 34.4 ราวหนึ่งในหกของ mark ที่คะแนนแบบไม่สนลำดับให้แก่โมเดลตั้งต้นมาจากสำเนาของการวนซ้ำ เราเปิดดู output ของทุกช่วงที่ถูกตัดแล้ว ทั้ง 126 ช่วงชนขีดจำกัด token ช่วงที่สั้นที่สุดมี 15 สำเนา และไม่มีข้อความปกติถูกตัดออก

mark ที่หายส่วนใหญ่ไม่ปรากฏใน output บนหน้าเต็ม Typhoon ทำ mark ผิด 6.2% (BQ) และ 7.1% (TC) ของ mark ในเฉลยทั้งหมด เราจัดแต่ละข้อผิดเข้าสาเหตุเดียวโดยอัตโนมัติ (รูปที่ 2) บรรทัดอาจถูกอ่านในลำดับอื่น คือมีช่วงของ output ตรงกับบรรทัดนั้นที่ character error rate ต่ำกว่า 20% (สำหรับบรรทัดที่ยาวอย่างน้อยแปดตัวอักษร) บรรทัดอาจหายไปทั้งบรรทัด ข้อความอาจขาดเป็นช่วงภายในบรรทัดที่เก็บไว้ หรือ mark อาจถูกอ่านผิดตัว บรรทัดที่หายทั้งบรรทัดคิดเป็น 2.3% (BQ) และ 2.9% (TC) ของ mark ข้อความที่ขาดเป็นช่วงคิดเป็น 0.7% และ 0.8% และการอ่านผิดตัวคิดเป็น 1.5% และ 2.4% ส่วนลำดับการอ่านอย่างเดียวคิดเป็น 1.6% และ 0.9% ซึ่งไม่ใช่ความผิดพลาดของการอ่าน เมื่อเปิดดู output จริง บรรทัดที่หายส่วนใหญ่เป็นข้อความใน infographic และกราฟ โฆษณาที่แทรกอยู่ รวมถึงชื่อเรื่องและส่วนประกอบของหน้า ซึ่ง Typhoon



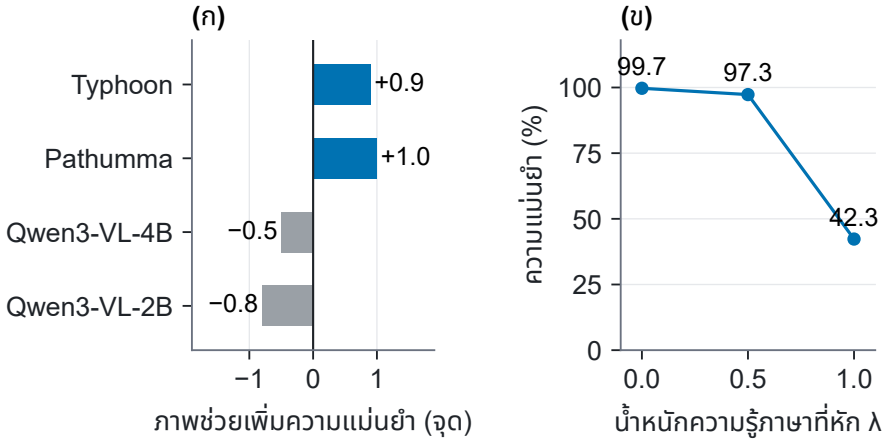
รูปที่ 2 mark บนหน้าเต็มที่หายไปกับสาเหตุใด คิดเป็นสัดส่วนของ mark ในเฉลยทั้งหมด สำหรับ output ที่ตัดที่การวนซ้ำครั้งแรก ยอดรวมของแต่ละแท่งไม่รวมส่วนสีเทา ซึ่งเป็นเพียงความต่างของลำดับการอ่านและไม่ใช่ข้อผิด

เขียนเป็นตัวแทนภาพไว้ สอดคล้องกับที่ผู้พัฒนาระบุเรื่อง infographic [2] การจัดสาเหตุเป็นแบบอัตโนมัติ และเราตรวจภาพหน้าเอกสารเฉพาะหกหน้าที่มี mark ที่หายเป็นส่วนใหญ่

รูปแบบเดียวกันพบในอีกห้าโมเดล (รูปที่ 2) เราจัดสาเหตุบน output ที่ตัดที่การวนซ้ำครั้งแรก เพราะ output ที่วนซ้ำยาวกว่าเฉลยหลายเท่า และการจัดแนวเชื่อถือไม่ได้ บรรทัดที่หายทั้งบรรทัดรวมกับข้อความที่ขาดเป็นช่วงทำให้เสีย mark 3.2% สำหรับ Typhoon 9.3% สำหรับ Pathumma และ 29 ถึง 44% สำหรับอีกสี่โมเดล ส่วนการอ่าน mark ผิดหลังอ่านพยัญชนะถูกทำให้เสีย 0.4% สำหรับ Typhoon 0.8% สำหรับ Pathumma และ 2.7 ถึง 4.3% สำหรับโมเดลที่เหลือ ในทุกโมเดล บรรทัดที่หายไปจึงทำให้เสีย mark มากกว่า mark ที่อ่านผิด 7 ถึง 12 เท่า การที่ข้อความหายจาก output คือสิ่งที่เราวัดได้ ซึ่งไม่เท่ากับโมเดลอ่านไม่ได้ เพราะโมเดลอาจเห็นบรรทัดแต่ไม่เขียนออกมา การแยกสองกรณีนี้ต้องใช้การทดสอบ เช่น ให้โมเดลอ่านบรรทัดนั้นเดี่ยว ๆ ในรูปที่ตัดมา ซึ่งเรายังไม่ได้ทำกับหน้าเต็ม สำหรับโมเดลที่วนซ้ำ บรรทัดที่อยู่หลังการวนซ้ำถูกนับว่าหายไป และเราแยกไม่ได้ว่าบรรทัดเหล่านั้นหายเพราะการวนซ้ำหรือเพราะโมเดลไม่อ่านอยู่แล้ว

เมื่อได้บริบทที่ถูกต้อง วรรณยุกต์แทบถูกเสมอ ในการทดสอบแบบเลือกคำตอบบังคับ เราป้อนข้อความที่ถูกต้องจนถึงตำแหน่งของ mark แล้วให้โมเดลให้คะแนนตัวเลือกที่เป็นไปได้ของ mark นั้นทั้งแบบมีภาพและไม่มีภาพ (fp32, tokenize ในบริบทเดียวกัน) บนหน้าเต็ม Typhoon เลือกวรรณยุกต์ถูก 99.7% เมื่อมีภาพ และ 98.8% เมื่อไม่มีภาพ ส่วนโมเดลตั้งต้นได้ 98.1% และ 98.9% ภาพจึงช่วยเพิ่ม 0.9 จุดให้ Typhoon และลดลง 0.8 จุดสำหรับโมเดลตั้งต้น (รูปที่ 3ก) การทดสอบนี้ให้บริบทที่ถูกต้องแก่โมเดล จึงไม่ใช่การ generate จริง

อีกสองโมเดล Pathumma ได้ 99.7% เมื่อมีภาพ และ 98.7% เมื่อไม่มีภาพ (ภาพช่วย +1.0) และได้ 94.8% เมื่อหักคะแนนแบบไม่มีภาพที่ $\lambda = 0.5$ ส่วน Qwen3-VL-4B รู้ mark ได้ดีเท่ากัน คือ 98.7% เมื่อมีภาพและภาพไม่ช่วย (99.1% เมื่อไม่มีภาพ) สองโมเดลต่างกันตอนเขียนหน้าเอง ในตำแหน่งที่ Pathumma อ่านพยัญชนะฐานถูก วรรณยุกต์ถูก 99.4% เทียบกับ 99.8% ของการทดสอบที่ตำแหน่งเดียวกัน ส่วน Qwen3-VL-4B ได้ 91.3% เทียบกับ 99.1% คืออ่านวรรณยุกต์ผิดราวหนึ่งตัวในสิบเอ็ดตัวตอน generate ทั้งที่การทดสอบแสดงว่ารู้ตัวที่ถูก ภาพช่วยเล็กน้อยกับ fine-tune ภาษาไทยสองตัว และไม่ช่วยโมเดลทั่วไป เราไม่ขยายความไปเกินกว่าสี่โมเดลนี้ สำหรับ Qwen3-VL-4B ข้อค้นพบว่า mark ส่วนใหญ่หายไปจาก output มากกว่าถูกอ่านผิดยังเป็นจริงในทิศทาง (บนชุด calibration ข้อความที่หายทำให้เสีย mark มากกว่าการอ่านผิดเหล่านี้ 7.3 เท่า)



รูปที่ 3 (ก) ความแม่นยำของวรรณยุกต์ที่เพิ่มขึ้นเมื่อมีภาพ ที่ตำแหน่งซึ่งลงทะเบียนไว้ (ชุด calibration) (ข) ความแม่นยำของ Typhoon เมื่อหักคะแนนแบบไม่มีภาพด้วยน้ำหนัก λ

แต่ไม่ใช่ข้อความที่ว่า mark แทบไม่เคยถูกอ่านผิด

ความมั่นใจของโมเดล ความน่าจะเป็นของ token ที่ Typhoon เขียนเองบอกตำแหน่งที่อ่าน mark ผิดได้ ค่าความน่าจะเป็นต่ำสุดของ token ภายในกลุ่มอักขระแยก mark ที่อ่านผิดออกจาก mark ที่ถูกด้วย AUROC 0.918 (BQ) และ 0.935 (TC) บน output แบบ greedy ของโมเดลเอง

5. Training-Free Decoding and Marks

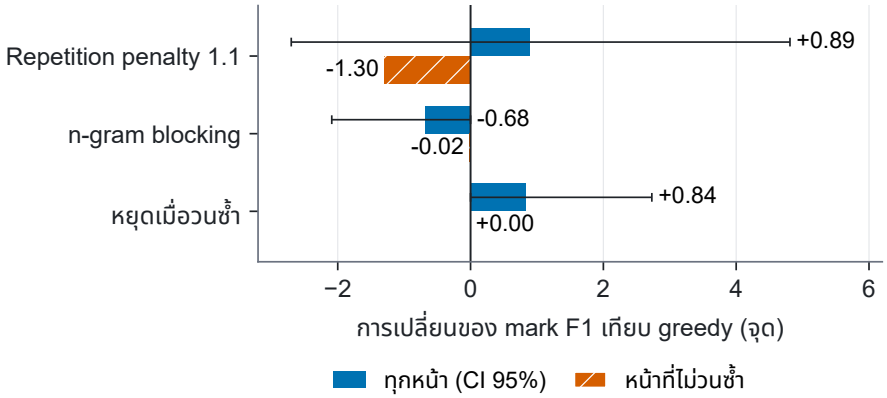
การวนซ้ำพบน้อยแต่เสียหายมาก จาก output ของ Typhoon 356 รายการ (ทั้งสอง prompt ทั้งสองงาน) มี 11 รายการที่วนซ้ำ ในแต่ละเซลล์ หน้าที่วนซ้ำเพียงสองถึงห้าหน้ามี mark ที่เขียนเกินรวมกัน 68 ถึง 76% ของทั้งหมด ถ้าไม่มีหน้าเหล่านี้ F1 บนหน้าเต็ม (BQ) จะเป็น 97.4 แทน 94.9 ตัวเลขนี้เป็นเพดานที่ได้จากการตัดหน้าเหล่านั้นออก ไม่ใช่ผลของวิธีใด

สามวิธีจัดการการวนซ้ำ เราเทียบวิธีจัดการการวนซ้ำสามวิธีสำหรับ Typhoon กับ greedy decoding (รูปที่ 4 หน้าเต็ม BQ)

- Repetition penalty 1.1 ซึ่งเป็นค่าต้านการวนซ้ำใน package ของ Typhoon เอง ใช้ penalty แบบเดียวกับ CTRL [18]
- n-gram blocking แบบเดียวกับโค้ดของ DeepSeek-OCR ซึ่งห้ามการซ้ำของ 30-gram ภายใน 90 token
- หยุดเมื่อตรวจพบการวนซ้ำ เมื่อหน่วยข้อความซ้ำแปดครั้ง หรือหกครั้งสำหรับหน่วยที่ยาว 50 ตัวอักษรขึ้นไป การ decode จะหยุดที่สำเนาแรก ผลของ greedy decoding ถูกกำหนดจาก prefix อยู่แล้ว เราจึงคำนวณจุดตัดแบบ offline จาก output greedy ที่ครบแล้ว เราตรวจการทำตอน decode จริงเทียบกับการตัดนี้ในหัวข้อ 6

Repetition penalty เปลี่ยน output เกือบทุกรายการ หน้าเต็ม BQ ที่ไม่วนซ้ำไม่มีหน้าใดเหมือน greedy เลย และ F1 ของหน้าที่ไม่วนซ้ำลดลง 1.1 ถึง 1.3 จุด ในงานอ่านข้อความสั้น (BQ) อัตราผิดของวรรณยุกต์เพิ่มจาก 17.1% เป็น 22.6% เมื่อรวมทุกหน้า ผลแยกจากศูนย์ไม่ได้ (+0.89 ช่วง −2.70 ถึง +4.81) ข้อความไทยมีพยางค์และวรรณยุกต์ซ้ำกันอยู่ตามปกติ ซึ่ง penalty ทั้งหน้าอาจกดไว้ แต่เรายังไม่ได้ทดสอบคำอธิบายนี้

n-gram blocking ปลอดภัยเมื่อไม่มีการวนซ้ำ output ที่ไม่วนซ้ำไม่เปลี่ยน 92 ถึง 100% และ F1 เปลี่ยนไม่เกิน 0.05 วิธีนี้หยุดการวนซ้ำได้ 6 จาก 11 รายการ ในรายการที่เหลือ โมเดลหนีไปวนแบบใกล้เคียงของ



รูปที่ 4 การเปลี่ยนแปลงของ mark F1 บนหน้าเต็มเทียบกับ greedy decoding (Typhoon, BQ) แท่งสีน้ำเงินคือทุกหน้าพร้อมช่วงความเชื่อมั่น 95% แท่งสีส้มคือเฉพาะหน้าที่ไม่วนซ้ำ

เดิม เช่น ชื่อซอยที่เพี้ยนไปเป็นชื่อซอยคล้ายกันสามชื่อ ข้อความเหล่านี้มี mark เพิ่ม ทำให้ mark ที่เขียนเกินสูงขึ้น และ F1 บนหน้าเต็มต่ำลง 0.68 สำหรับ BQ เราตีความว่าการวนซ้ำเป็นสถานะดึงดูดของ decoder การห้ามสำเนาที่ซ้ำตรงตัวจึงย้ายโมเดลไปอยู่ในสถานะข้างเคียง นี่เป็นการตีความ ยังไม่ได้ทดสอบ

หยุดเมื่อพบการวนซ้ำ ไม่เปลี่ยน output ที่ไม่วนซ้ำเลยในทุกเซลล์ mark F1 บนหน้าเต็มเพิ่มขึ้น 0.84 (BQ) และ 0.67 (TC) และ F1 ของงานอ่านข้อความสั้นเพิ่มขึ้น 3.30 และ 2.39 การหยุดยังประหยัด token ที่สร้างได้ 11.2% ส่วน recall ลดลงไม่เกิน 0.16 จุด เพราะการหยุดกู้ข้อความที่การวนซ้ำทำให้ข้ามไปไม่ได้ เราเขียนกฎนี้หลังจากเห็นผลการเปรียบเทียบก่อนหน้า และแต่ละเซลล์มี output ที่วนซ้ำเพียงหนึ่งถึงสี่รายการ ช่วงความเชื่อมั่นทุกช่วงจึงรวมศูนย์ไว้

การหยุดกับโมเดลอื่น โมเดลอื่นวนซ้ำบ่อยกว่า Typhoon มาก บนหน้าเต็ม output ของโมเดลตั้งต้น Qwen3-VL-4B Paddle และ Wayu วนซ้ำ 18 ถึง 30 จาก 69 รายการ เทียบกับ 2 รายการของ Typhoon และ 4 รายการของ Pathumma เมื่อใช้การหยุดแบบ offline เดียวกันกับ output greedy ของโมเดลเหล่านี้ mark F1 เพิ่มขึ้นในทุกโมเดลทั้งสองงาน ไม่มีช่วงความเชื่อมั่นใดต่ำกว่าศูนย์ และเจ็ดจากสิบสองเซลล์ไม่รวมศูนย์บนหน้าเต็ม ผลที่เพิ่มมากในโมเดลที่วนซ้ำบ่อย เช่น +24.4 สำหรับโมเดลตั้งต้น และ +10.8 สำหรับ Wayu (ช่วง 5.3 ถึง 16.1) และน้อยในโมเดลที่แทบไม่วนซ้ำ คือ +0.8 สำหรับ Typhoon และ +3.0 สำหรับ Pathumma หัวข้อ 6 ทดสอบซ้ำบนข้อมูลที่ไม่เคยเห็นกับทั้งแปดโมเดล

ผลที่เพิ่มมาจาก precision และแลกกับ recall ที่ลดลงสูงสุด 10.6 จุดบนหน้าเต็ม recall ส่วนหนึ่งเป็นคะแนนที่สำเนาของการวนซ้ำเคยได้ และที่เหลือคือข้อความหลังการวนซ้ำที่ถูกตัดทิ้ง ในสิบสองเซลล์ การหยุดเปลี่ยน output ที่จบเองเพียง 3 จาก 898 รายการ สองรายการเป็น output ของ Paddle ที่ออกจากการวนซ้ำเองแล้ว และการตัดทำให้ข้อความจริงของหน้าที่ตามหลังหายไป repetition penalty 1.05 เพียงอย่างเดียว ซึ่งเป็นค่าในตัวอย่างของ Wayu เอง ไม่ช่วย (+2.8 ช่วง −3.6 ถึง +9.6 บนหน้าเต็ม และ −11.3 ช่วง −22.5 ถึง +4.5 บนงานอ่านข้อความสั้น)

การหยุดประหยัด token ที่สร้างได้ 34% บนหน้าเต็มของ Wayu ในการรันนี้ 72% ของช่องว่าง recall ระหว่าง Wayu กับ Typhoon บนหน้าเต็มอยู่ใน 18 หน้าที่ Wayu วนซ้ำ และไม่มีวิธี decode ใดที่เราทดสอบกู้คืนได้ เรายังบอกไม่ได้ว่าการวนซ้ำเป็นสาเหตุหรือเป็นอาการ

การหักความรู้ภาษาออกทำให้ mark ที่ถูกหายไปด้วย ในการทดสอบแบบเลือกคำตอบบังคับ การหักคะแนนแบบไม่มีภาพด้วยน้ำหนัก λ ทำให้ความแม่นยำของวรรณยุกต์ของ Typhoon ลดจาก 99.7% เป็น 97.3% ที่ $\lambda = 0.5$ และเหลือ 42.3% ที่ $\lambda = 1$ (รูปที่ 3ข) ส่วนโมเดลตั้งต้นลดเหลือ 84.7% ที่ 0.5 ในการ generate

จริง บนกลุ่มย่อย 24 ข้อ contrast แบบ M3ID-form ของเราหยุดการวนซ้ำของโมเดลตั้งต้นได้ (character error rate จาก 1.12 เหลือ 0.21) แต่ก็ทำให้อัตราผิดของวรรณยุกต์ในข้อที่เก็บไว้เพิ่มจาก 3.7% เป็น 7.6% ส่วนใหญ่เพราะวรรณยุกต์ถูกลบ (21 เป็น 55 ใน 12 ข้อ) ส่วน Typhoon เปลี่ยนเพียงเล็กน้อยและกระจุกอยู่ในไม่กี่ข้อ ผลทั้งสองชี้ไปทางเดียวกัน ความรู้ภาษาเกี่ยวกับวรรณยุกต์ถูกอยู่แล้ว การหักออกจึงลบวรรณยุกต์ที่ถูกไปด้วย

ความเร็ว Speculative decoding แบบ prompt lookup ทำให้ Typhoon เร็วขึ้น 1.14 ถึง 1.15 เท่า (โมเดลตั้งต้น 1.16 ถึง 1.21 เท่า) บนหน้าที่ไม่เสื่อมเป็นการวนซ้ำ output เหมือนเดิมทุกตัวอักษรยกเว้นจุดที่คะแนนของสองตัวเลือกเกือบเท่ากันใน fp16 โดย 88 ถึง 93% ของทุกข้อเหมือนเดิม ตัวอ้างอิง greedy ให้ output ตรงกับ output หลักของเราทุกตัวอักษรในทั้ง 177 ข้อที่จับเวลา เรายังไม่ได้วัดความเร็วเมื่อใช้การหยุดร่วมกับ speculative decoding หรือต้นทุนของการตรวจการวนซ้ำทุก token

6. Confirmation on the Locked Split

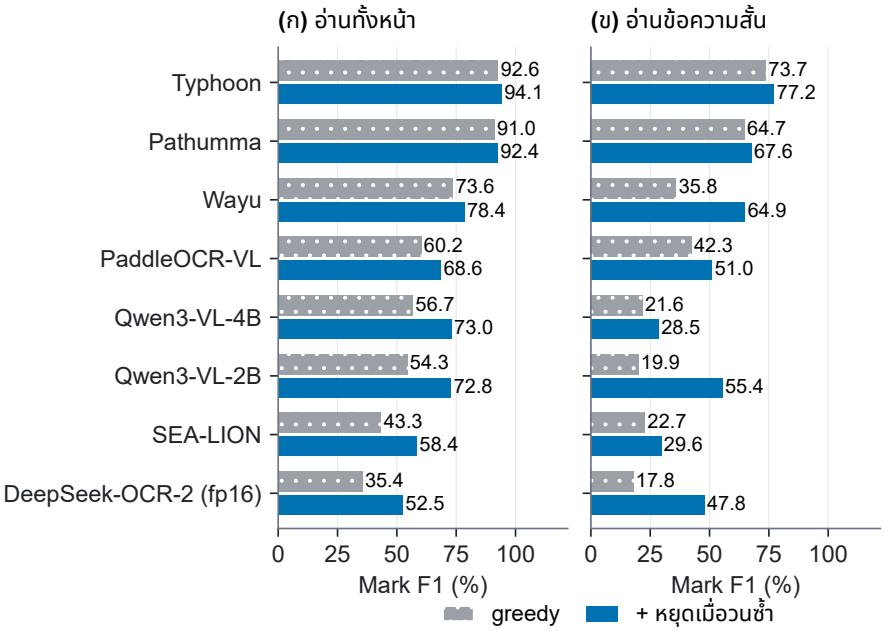
ทุกอย่างข้างต้นพัฒนาขึ้นบนชุด calibration จากนั้นเราลงทะเบียนการทดสอบสามข้อ พร้อมกำหนดวิธีอ่านผลไว้ล่วงหน้า และเปิดอ่านชุด locked เพียงครั้งเดียว คือ 352 ข้อ (งานอ่านทั้งหน้า 128 ข้อ งานอ่านข้อความสั้น 224 ข้อ) กับแปดโมเดล ได้แก่หกโมเดลข้างต้น บวก Qwen-SEA-LION-v4-4B-VL และ DeepSeek-OCR-2 prompt การ decode แบบ greedy และ fp16 เป็นไปตามหัวข้อ 3 และการตัดที่การวนซ้ำครั้งแรกทำแบบ offline กับ output greedy จากการรันเดียวกัน การทดสอบมีดังนี้

- S1: สำหรับแต่ละโมเดลและงานที่โมเดลวนซ้ำอย่างน้อย 5% ของข้อใน calibration การหยุดเพิ่ม mark F1 มีเซลล์แบบนี้ 13 เซลล์
- S2: การหยุดเปลี่ยน output ที่จบเองไม่เกิน 1% เมื่อรวมทุกโมเดลและทุกงาน
- R1: สำหรับแต่ละโมเดลบนหน้าเต็ม mark ที่เสียไปกับบรรทัดที่หายและข้อความที่ขาดเป็นช่วงมากกว่า mark ที่เสียไปกับการอ่านผิดหลังอ่านพยัญชนะถูกเกินสามเท่า โดยขอบล่างของช่วงความเชื่อมั่น 95% ต้องสูงกว่าสาม

การแก้ Holm หนึ่งชุดครอบคลุมทั้ง 22 การทดสอบ (S1 13 ข้อ S2 หนึ่งข้อ และ R1 แปดข้อ) ช่วงความเชื่อมั่นเป็น bootstrap รายหน้า 10,000 รอบ ค่า p แบบทางเดียวที่เล็กที่สุดที่ทำได้คือ 0.0001 ค่าที่ปรับแล้ว 0.0021 จึงอ่านว่า “ไม่เกิน 0.0021”

การหยุด mark F1 เพิ่มขึ้นใน 12 จาก 13 เซลล์ที่เข้าเกณฑ์ (รูปที่ 5 และตารางที่ 2) เพิ่ม 4.8 ถึง 35.6 จุด โดยค่า p ที่ปรับแล้วไม่เกิน 0.0036 เซลล์ที่สิบสาม คือ Pathumma บนหน้าเต็ม ไม่ผ่าน ผลที่เพิ่มคือ +1.3 จุด (ช่วง 0.0 ถึง +3.5, p ที่ปรับแล้ว 0.13) เมื่อให้คะแนนซ้ำหลังเห็นผลโดยแก้ข้อบกพร่องการสกัดข้อความสองจุดที่อธิบายด้านล่าง เซลล์นี้ได้ +2.2 (ช่วง 0.0 ถึง +4.9, p ที่ปรับแล้ว 0.046) แปลว่าผลของเซลล์นี้ไวต่อการสกัดข้อความ ผลที่ลงทะเบียนไม่เปลี่ยน เซลล์นี้เข้าเกณฑ์บน calibration ด้วยระยะที่ฉิวเฉียด (วนซ้ำ 4 จาก 69 หน้า) และวนซ้ำ 4 จาก 128 หน้าบนชุด locked (3.1%) ซึ่งเป็นอัตราของเซลล์ที่เราไม่ได้ทดสอบ โมเดลที่แทบไม่วนซ้ำจึงได้ผลเพิ่มเล็กน้อย และที่ขนาดตัวอย่างนี้ยังแสดงไม่ได้ เกณฑ์กำหนดให้ทุกเซลล์ที่เข้าเกณฑ์ต้องผ่าน S1 ตามที่ลงทะเบียนไว้จึงไม่ผ่าน และเราระบุเซลล์นี้ไว้ S2 ผ่าน การหยุดเปลี่ยน output ที่จบเอง 1 จาก 2,400 รายการ (0.04% ขอบบน 95% คือ 0.20%) ซึ่งเป็น output ของ Typhoon ที่ออกจากการวนซ้ำจริงด้วยตัวเอง

mark หายที่ไหน R1 ผ่านทั้งแปดโมเดล (ตารางที่ 2) บนหน้าเต็ม ข้อความที่หายทำให้เสีย mark มากกว่า mark ที่อ่านผิดหลังอ่านพยัญชนะถูก 4.7 เท่า (PaddleOCR-VL ช่วง 3.2 ถึง 6.7) ถึง 15.8 เท่า (Pathumma) ขอบ



รูปที่ 5 Mark F1 บนชุด locked เมื่อไม่หยุดและเมื่อหยุดที่การวนซ้ำ (ก) งานอ่านทั้งหน้า (ข) งานอ่านข้อความสั้น prompt ตามที่ลงทะเบียนไว้ DeepSeek-OCR-2 เป็น fp16 เท่านั้น

ล่างของ PaddleOCR-VL ใกล้เกณฑ์มากที่สุด ข้อนี้เป็นข้อความเกี่ยวกับแปดโมเดลนี้บน benchmark เดียว และเช่นเดิม สิ่งที่เราวัดคือการที่ข้อความไม่ปรากฏใน output ซึ่งไม่เท่ากับโมเดลอ่านไม่ได้

การจัดอันดับ mark F1 ของ Typhoon สูงกว่าหกโมเดลที่ไม่ใช่ Pathumma ทั้งสองงาน โดยช่วงความเชื่อมั่นอยู่เหนือศูนย์ เทียบกับ Pathumma ไม่พบความต่าง เมื่อใช้ prompt ของ model card ของ Pathumma ส่วนต่างคือ +1.5 จุดบนหน้าเต็ม (ช่วง −3.1 ถึง +6.5) และ +9.0 บนงานอ่านข้อความสั้น (ช่วง −3.6 ถึง +21.3) ที่ 128 และ 224 ข้อ ครึ่งความกว้างของช่วงอยู่ที่ราว 6 และ 13 จุด จึงไม่ใช่ความเท่ากัน และเราไม่เรียกโมเดลใดว่าดีที่สุด การสกัดข้อความมีข้อบกพร่องสองจุดกับ output หน้าเต็มของ Pathumma คือแท็ก `<figure>` ที่ไม่ถูกปิดทำให้ข้อความของหน้าหลังแท็กหายไป (เก้ารายการจากสอง prompt) และคำตอบในรูป JSON ของ Typhoon เก็บการขึ้นบรรทัดไว้เป็นตัวอักษร `\n` จริง (30 รายการ) เมื่อให้คะแนนซ้ำโดยแก้ทั้งสองจุด (ทำหลังเห็นผล) ส่วนต่างบนหน้าเต็มกับ prompt ของ card เป็น +2.4 (ช่วง −2.6 ถึง +7.8) ข้อสรุปไม่เปลี่ยน F1 บนชุด locked ของ Typhoon (92.6 และ 73.7) ต่ำกว่า F1 บน calibration (94.9 และ 81.7) เราไม่ได้ปรับอะไรกับข้อใน locked จึงคาดว่าจะลดลงเช่นนี้บนหน้าที่ไม่เคยเห็น และเราใช้ตัวเลขของชุด locked เป็นตัวเลขหลัก

การหยุดตอน generate จริง output บนชุด locked ถูกตัดแบบ offline เพื่อตรวจกฎตอน decode เรา generate ใหม่บนข้อใน calibration โดยต่อตัวหยุดเข้ากับการ generate ครบแปดโมเดล คือ 185 ข้อที่การตัดแบบ offline เปลี่ยน output และ 72 ข้อที่สุ่มมาจากข้อที่การตัดไม่แตะ ผลคือ output ที่หยุดแล้วเท่ากับการตัดแบบ offline ทุกตัวอักษรใน 185 จาก 185 ข้อ และกฎไม่ทำงานเลยใน 72 จาก 72 ข้อที่การตัดไม่แตะ ส่วน output ที่วนซ้ำใช้ token 15.3% ของที่เคยใช้ การตรวจนี้ครอบคลุมเฉพาะข้อใน calibration และเรายังไม่ได้วัดเวลาที่ประหยัดได้

7. Discussion, Limitations and Conclusion

บนชุด locked แปดโมเดล และบนชุด calibration หกโมเดล mark ที่เสียไปส่วนใหญ่เสียก่อนจะถึงตัว mark ได้แก่ บรรทัดที่หายจาก output ข้อความที่ขาดเป็นช่วง และการวนซ้ำ เมื่อได้ข้อความที่ถูกต้อง วรรณยุกต์แทบ

ตารางที่ 2 ชุด locked การเปลี่ยนของ mark F1 เมื่อหยุดที่การวนซ้ำ (จุด ช่วงความเชื่อมั่น 95%) และ R1 คืออัตราส่วนของ mark ที่เสียไปกับข้อความที่หาย ต่อ mark ที่เสียไปกับการอ่านผิด (หน้าเต็ม) †ไม่ได้ทดสอบ เพราะโมเดลวนซ้ำน้อยกว่า 5% ของข้อใน calibration ‡ไม่ผ่านการทดสอบที่ลงทะเบียนไว้

โมเดล	อ่านทั้งหน้า ΔF1	อ่านข้อความสั้น ΔF1	R1
Typhoon OCR 1.5	+1.5 [−0.5, +4.1]†	+3.5 [0.0, +11.0]†	8.2 [5.5, 14.3]
Pathumma 3.0	+1.3 [0.0, +3.5]‡	+3.0 [0.0, +6.7]†	15.8 [7.5, 32.0]
Wayu	+4.8 [+1.8, +8.3]	+29.1 [+18.7, +38.0]	10.8 [7.1, 16.0]
PaddleOCR-VL-1.6	+8.4 [+4.5, +12.0]	+8.8 [+2.2, +14.6]	4.7 [3.2, 6.7]
Qwen3-VL-4B	+16.3 [+11.7, +20.3]	+7.0 [+4.2, +10.1]	6.1 [3.7, 9.2]
Qwen3-VL-2B	+18.4 [+14.3, +22.2]	+35.6 [+29.6, +40.9]	7.8 [5.0, 11.7]
SEA-LION v4 4B	+15.2 [+11.9, +18.4]	+6.9 [+3.8, +10.0]	7.8 [5.6, 10.4]
DeepSeek-OCR-2 (fp16)	+17.1 [+12.8, +21.5]	+30.0 [+21.8, +36.3]	7.8 [5.8, 10.5]

แน่นอนสำหรับ fine-tune ภาษาไทยสองตัว และวิธีที่หักความรู้ภาษาออกต้องแลกด้วยวรรณยุกต์ วิธีที่ถูกและปลอดภัยในการจัดการการวนซ้ำคือหยุดมัน และผลที่ได้มากเมื่อโมเดลวนซ้ำบ่อย ภาพรวมนี้ไม่เหมือนกันทุกโมเดล Qwen3-VL-4B อ่าน mark ผิดตอนเขียนหน้าเองด้วย

ขอบเขต งานนี้ใช้ benchmark เดียวและสองงาน และโมเดลเป็น checkpoint แบบเปิดที่ศึกษาตามที่เผยแพร่ ซึ่งจำนวน visual token ไม่เท่ากัน โมเดลที่ออกแบบให้อ่านทีละกรอบสองตัวถูกนำมาอ่านทั้งหน้าเกินกว่าที่ออกแบบไว้ และ DeepSeek-OCR-2 รันด้วย fp16 อย่างเดียวและใช้ runner ของตัวเอง Typhoon มาจากกลุ่มที่เผยแพร่ benchmark และไม่ระบุข้อมูลฝึกของ Pathumma เราจึงตัดความเป็นไปได้ที่ข้อมูลซ้อนกันไม่ได้และไม่จัดอันดับสองโมเดลนี้ output ที่วนซ้ำของ Typhoon และ Pathumma มีน้อย ช่วงความเชื่อมั่นส่วนใหญ่จึงรวมศูนย์ไว้ ข้อสรุปของเราจึงเป็นเรื่องกลไก ไม่ใช่อัตรา

การออกแบบ หัวข้อ 4 และ 5 มาจากชุด calibration และกฎการหยุด คะแนนแบบกันการวนซ้ำ และการให้คะแนนถูกเขียนหลังจากเห็นข้อมูลชุดนั้นแล้ว หัวข้อ 6 คือการยืนยัน ซึ่งลงทะเบียนไว้ก่อนให้คะแนน output ของชุด locked ใดๆ และเปิดอ่านครั้งเดียว ตัวเลขที่ควรอ้างจึงเป็นตัวเลขจากหัวข้อนี้ ผลนี้สนับสนุนการวินิจฉัยและการหยุดสำหรับแปดโมเดลนี้ และเราไม่อ้างเกินกว่านั้น S1 ตามที่ลงทะเบียนไว้ไม่ผ่านเพราะหนึ่งเซลล์ บนชุด locked การหยุดทำแบบ offline และการตรวจตอน decode ครอบคลุมเฉพาะข้อใน calibration ในงานอ่านข้อความสั้น หลายโมเดลมี precision ต่ำเพราะข้อแสดงทั้งหน้าแต่คำถามถามเพียงบางส่วน ซึ่งเรายังไม่ได้ทดสอบ

การตีความ การจัดสาเหตุของข้อผิดเป็นแบบอัตโนมัติ การที่ข้อความไม่ปรากฏใน output ไม่เท่ากับโมเดลอ่านไม่ได้ และการจัดสาเหตุหลังการตัดแยกการวนซ้ำออกจากการที่โมเดลไม่เขียนข้อความนั้นเองไม่ได้ มีหลักฐานทางอ้อมว่าข้อความที่ไม่ปรากฏส่วนหนึ่งโมเดลอ่านได้ ในงานนำร่องแยกต่างหากกับโมเดลเดียวบนคำถามเกี่ยวกับกรอบข้อความ 69 ข้อ Typhoon พบข้อความในกรอบอยู่ในคำตอบ 7.2% ของข้อเมื่อได้ภาพทั้งหน้า (33.3% เมื่อคำถามไม่ระบุระบบพิกัด) และ 98.6% เมื่อได้เฉพาะภาพที่ตัดมา งานนำร่องนี้ถามเกี่ยวกับกรอบ ไม่ใช่ถอดทั้งหน้า ใช้โมเดลเดียว และไม่ได้บอกว่าอธิบายบรรทัดที่หายของเราได้กี่ส่วน การหยุดบางครั้งตัดข้อความจริงที่ตามหลังการวนซ้ำ (3 จาก 898 output ที่จบเองบนชุด calibration และ 1 จาก 2,400 บนชุด locked) การทดสอบแบบเลือกคำตอบบังคับใช้บริบทที่ถูกต้อง จึงแสดงว่าโมเดลรู้อะไร ไม่ใช่โมเดลทำอะไรตอน generate และ contrast ที่เราใช้เป็นการเขียนใหม่ที่ทดสอบบน 24 ข้อ สุดท้าย เราใช้ revision ของ Typhoon ที่เผยแพร่ภายใต้ Apache-2.0 อย่างเดียว ขณะที่เงื่อนไขปัจจุบันของผู้พัฒนาขอให้ได้รับความยินยอมเป็นลายลักษณ์อักษรก่อนนำไปเปรียบเทียบประสิทธิภาพ

สรุป OCR ภาษาไทยแบบเปิดที่เก่งไม่ได้พลาดที่วรรณยุกต์เป็นหลัก บนชุด locked ในทั้งแปดโมเดลที่

ทดสอบ mark ที่เสียไปส่วนใหญ่เสียก่อนหน้านั้น เพราะข้อความไม่ปรากฏใน output คือโมเดลหาหรือเขียนข้อความนั้นไม่ได้ หรือวนซ้ำ เมื่อได้บริบทที่ถูกต้อง fine-tune ภาษาไทยสองตัวเลือกวรรณยุกต์ถูกราว 99% แม้โมเดลทั่วไปขนาด 4B ยังอ่าน mark ผิดบางตัวตอนเขียน การหยุดเมื่อตรวจพบการวนซ้ำเป็นขั้นตอนที่ไม่ต้องฝึกโมเดลเพิ่ม เพิ่ม mark F1 ใน 12 จาก 13 เซลล์ที่ลงทะเบียนไว้ และแทบไม่เปลี่ยน output ที่จบเอง ส่วนการหักความรู้ภาษาออกไม่ใช่ทางที่ดีในการกู้ mark คืน ผลทั้งหมดมาจาก benchmark เดียว

Ethical and Licensing Notes

โมเดลทั้งหมดเป็นโมเดลแบบเปิดและใช้ตามสัญญาอนุญาตของแต่ละตัว (Apache-2.0 หรือ MIT โดย Typhoon ใช้ revision ที่เป็น Apache-2.0 อย่างเดียว) ThaiOCRBench เผยแพร่ภายใต้ CC-BY-SA-4.0 เราอ้างอิงบทความและ revision ของ benchmark รายงาน output และคะแนน และไม่เผยแพร่ภาพของ benchmark ซ้ำ

เอกสารอ้างอิง

[1] S. Nonesung, T. Jaknamon, S. Chaiophat, N. Nitarach, C. Wittayasakpan, W. Sirichotedumrong, A. Na-Thalang, and K. Pipatanakul, "ThaiOCRBench: A task-diverse benchmark for vision-language understanding in Thai," in *Proceedings of IJCNLP-AACL 2025 (Long Papers)*, 2025, pp. 1653–1675, arXiv:2511.04479. [Online]. Available: https://aclanthology.org/2025.ijcnlp-long.89

[2] S. Nonesung, N. Nitarach, T. Jaknamon, P. Taveekitworachai, and K. Pipatanakul, "Typhoon OCR: Open vision-language model for Thai document extraction," 2026. [Online]. Available: https://arxiv.org/abs/2601.14722

[3] NECTEC, "Pathumma-llm-vision-3.0.0-re," Hugging Face model card, 2026, revision 08bb11dcabf509ac3fa3ac0014ecd44eadb9729b. [Online]. Available: https://huggingface.co/nectec/Pathumma-llm-vision-3.0.0-re

[4] K. Pipatanakul, "How far can synthetic data take Thai OCR?" 2026. [Online]. Available: https://arxiv.org/abs/2609.03595

[5] S. Leng, H. Zhang, G. Chen, X. Li, S. Lu, C. Miao, and L. Bing, "Mitigating object hallucinations in large vision-language models through visual contrastive decoding," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024, pp. 13 872–13 882.

[6] A. Favero, L. Zancato, M. Trager, S. Choudhary, P. Perera, A. Achille, A. Swaminathan, and S. Soatto, "Multi-modal hallucination control by visual information grounding," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024, arXiv:2403.14003.

[7] S. Bai, Y. Cai, R. Chen *et al.*, "Qwen3-VL technical report," 2025. [Online]. Available: https://arxiv.org/abs/2511.21631

[8] C. Cui, T. Sun, S. Liang *et al.*, "PaddleOCR-VL: Boosting multilingual document parsing via a 0.9B ultra-compact vision-language model," 2025. [Online]. Available: https://arxiv.org/abs/2510.14528

[9] T. Do, D. P. Tran, A. Vo, and D. Kim, "Reference-based post-OCR processing with LLM for precise diacritic text in historical document recognition," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, no. 27, 2025, pp. 27 951–27 959.

[10] H. Elsafty, F. Aldabbas, and R. Sifa, "Arabic OCR in the age of multimodal models: A comprehensive comparative evaluation," in *2025 IEEE International Conference on Big Data*, 2025, pp. 5437–5442, lamarr Institute publication page.

[11] A. Holtzman, J. Buys, L. Du, M. Forbes, and Y. Choi, "The curious case of neural text degeneration," in *International Conference on Learning Representations (ICLR)*, 2020, arXiv:1904.09751.

[12] L. Blecher, G. Cucurull, T. Scialom, and R. Stojnic, "Nougat: Neural optical understanding for academic documents," in *International Conference on Learning Representations (ICLR)*, 2024, arXiv:2308.13418.

[13] J. Poznanski, A. Rangapur, J. Borchardt, J. Dunkelberger, R. Huff, D. Lin, C. Wilhelm, K. Lo, and L. Soldaini, "olmOCR: Unlocking trillions of tokens in PDFs with vision language models," 2025. [Online]. Available: https://arxiv.org/abs/2502.18443

[14] PaddlePaddle, "PaddleX PaddleOCR-VL pipeline: `truncate_repetitive_content`," GitHub source code, 2025. [Online]. Available: https://github.com/PaddlePaddle/PaddleX/blob/ffb64904d23708863ff5b8da312a5cbd52a7f462/paddlex/inference/pipelines/paddleocr_vl/uilts.py

[15] DeepSeek-AI, "DeepSeek-OCR source code: `NoRepeatNGramLogitsProcessor`," GitHub, 2025. [Online]. Available: https://github.com/deepseek-ai/DeepSeek-OCR

[16] H. Wei, Y. Sun, and Y. Li, "DeepSeek-OCR: Contexts optical compression," 2025. [Online]. Available: https://arxiv.org/abs/2510.18234

[17] Z. Yang, J. Tang, Z. Li, P. Wang, J. Wan, H. Zhong, X. Liu, M. Yang, P. Wang, S. Bai, L. Jin, and J. Lin, "CC-OCR: A comprehensive and challenging OCR benchmark for evaluating large multimodal models in literacy," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2025, pp. 21 744–21 754.

[18] N. S. Keskar, B. McCann, L. R. Varshney, C. Xiong, and R. Socher, "CTRL: A conditional transformer language model for controllable generation," 2019.

[19] S. Liu, K. Zheng, and W. Chen, "Paying more attention to image: A training-free method for alleviating hallucination in LVLMs," in *European Conference on Computer Vision (ECCV)*, 2024.

[20] Y. Leviathan, M. Kalman, and Y. Matias, "Fast inference from transformers via speculative decoding," in *International Conference on Machine Learning (ICML)*, 2023, arXiv:2211.17192.

[21] A. Saxena, "Prompt lookup decoding," GitHub, 2023. [Online]. Available: https://github.com/apoorvumang/prompt-lookup-decoding

[22] AI Singapore, "Qwen-sea-lion-v4-4b-vl," Hugging Face model card, 2026, revision d0734aa1b8113fce90997a83c1e4afe2c37c6205. [Online]. Available: https://huggingface.co/aisingapore/Qwen-SEA-LION-v4-4B-VL

[23] H. Wei, Y. Sun, and Y. Li, “DeepSeek-OCR 2: Visual causal flow,” 2026, model: deepseek-ai/DeepSeek-OCR-2, revision aaa02f3811945a91062062994c5c4a3f4c0af2b0. [Online]. Available: https://arxiv.org/abs/2601.20552